

# OpenSooq CDC Data Analyzer

---

Convert complicated CDC and raw technical listing data into clear, readable business information — processed entirely in the browser.

---

**Product** · opensooq-cdc-analyzer v0.1.0

**Author** · Hashem Matouq

**Organization** · OpenSooq

**Stack** · Next.js 15 · React 19 · TypeScript · Tailwind CSS 4

**Document date** · August 2026

**Audience** · Engineers, QA, product, and operations teams

# Table of Contents

---

|           |                                 |
|-----------|---------------------------------|
| <b>01</b> | Executive Overview              |
| <b>02</b> | Goals & Value Proposition       |
| <b>03</b> | System Architecture             |
| <b>04</b> | Repository Structure            |
| <b>05</b> | Technology Stack                |
| <b>06</b> | Getting Started                 |
| <b>07</b> | User Guide — Features & Screens |
| <b>08</b> | Data Pipeline                   |
| <b>09</b> | Field Dictionary                |
| <b>10</b> | Listing Status Codes            |
| <b>11</b> | Insights, Lifecycle & QA        |
| <b>12</b> | State Management                |
| <b>13</b> | Export & Formatting             |
| <b>14</b> | Privacy & Security              |
| <b>15</b> | Sample Data                     |
| <b>16</b> | Type System Reference           |
| <b>17</b> | Build, Deploy & Operations      |
| <b>18</b> | Troubleshooting                 |
| <b>19</b> | Appendix — Glossary & Credits   |

# 1. Executive Overview

---

**OpenSooq CDC Data Analyzer** is a client-side web application that turns Change Data Capture (CDC) exports — typically dense database dumps in CSV or PDF form — into human-readable business intelligence about OpenSooq listings.

Operations, moderation, and engineering teams often receive CDC snapshots that contain opaque field names ( `SF_OP` , `SF_OFFSET_ID` , `LAST_EDIT_ACTION` ), numeric status codes, and rule-engine JSON. This product maps that raw stream into:

- A dashboard of activity, status, and risk signals
- Searchable, filterable record tables with human or raw labels
- Full listing lifecycle timelines (first event → last event, with gaps and evidence)
- Status history tracking across official OpenSooq status codes (100–301)
- Smart insights and QA anomaly detection
- Side-by-side record comparison and multi-format export

## Privacy by design

All parsing and analysis run locally in the user's browser. Uploaded files are never sent to a backend or third-party AI service by default.

## 2. Goals & Value Proposition

### 2.1 Problems solved

| Problem                                                  | How the analyzer helps                                                        |
|----------------------------------------------------------|-------------------------------------------------------------------------------|
| CDC dumps are hard to read                               | Field dictionary maps technical names to business labels and descriptions     |
| Status codes are numeric and inconsistent across sources | Official codes 100–301 plus aliases (including Arabic-aware normalization)    |
| Listing journeys are fragmented across many rows         | Lifecycle replay groups events by listing with chronological ordering         |
| Anomalies are easy to miss                               | QA engine flags multi-status churn, post-delete updates, missing prices, etc. |
| Sensitive PII appears in exports                         | Phone, email, and member ID masked by default with optional reveal            |
| Data cannot leave the workstation                        | 100% browser-side processing; no upload to servers                            |

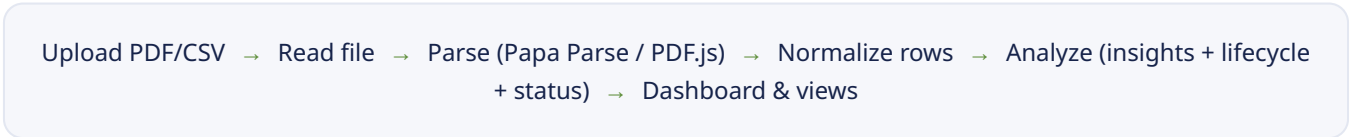
### 2.2 Primary users

|                                                                                                                             |                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Operations &amp; Moderation</b><br>Investigate listing status journeys, violations, and rule-engine actions without SQL. | <b>Engineering &amp; QA</b><br>Validate CDC exports, spot ordering issues, compare records, and export evidence packs. |
|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|

### 3. System Architecture

The application is a single-page Next.js client. There is no application backend for file processing. The browser loads the UI, the user selects a file, and the analysis pipeline runs entirely in memory via Zustand state.

#### 3.1 High-level flow



#### 3.2 Processing steps (UI)

| Step ID   | Label                    | What happens                                        |
|-----------|--------------------------|-----------------------------------------------------|
| read      | Reading file             | File bytes/text loaded in the browser               |
| extract   | Extracting records       | CSV or PDF parser produces raw row objects          |
| detect    | Detecting CDC operations | SF_OP / aliases mapped to insert/update/delete      |
| map       | Mapping fields           | normalizeRawRow builds structured CDCRecord s       |
| insights  | Generating insights      | analyzeRecords builds stats, lifecycles, QA, charts |
| dashboard | Preparing dashboard      | Navigate to dashboard with full AnalysisResult      |

#### 3.3 Architectural principles

- **Client-only analysis** — no server round-trip for CDC content
- **Evidence-first UI** — every insight/timeline step carries evidence strings and an evidence category
- **Offset-aware chronology** — prefer SF\_OFFSET\_ID over timestamps when ordering events
- **Human / raw dual view** — same data, two presentation modes
- **OpenSooq brand fidelity** — official palette (blue #33529D , green #87BE57 )

## 4. Repository Structure

```
/workspace
├─ package.json          # Dependencies & scripts
├─ next.config.ts        # Next.js config (PDF.js canvas alias)
├─ tsconfig.json         # TypeScript (@/* alias)
├─ postcss.config.mjs    # Tailwind CSS v4
├─ eslint.config.mjs
├─ README.md
├─ public/
│   └─ opensooq-logo.png # Brand logo / favicon
├─ sample-data/
│   └─ sample-cdc.csv    # Demo CDC export (16 rows, 6 listings)
├─ docs/
│   └─ OpenSooq-CDC-Data-Analyzer-Documentation.pdf
└─ src/
    ├─ app/
    │   ├─ layout.tsx    # Root layout, fonts, metadata
    │   ├─ page.tsx      # Entry → AppRoot
    │   └─ globals.css   # Design tokens & animations
    ├─ types/cdc.ts      # Core TypeScript contracts
    ├─ config/
    │   ├─ field-definitions.ts # 40+ field dictionary entries
    │   └─ listing-statuses.ts # STATUS codes 100-301
    ├─ lib/
    │   ├─ store.ts       # Zustand store + processFile
    │   ├─ utils.ts       # Helpers (cn, formatBytes, keys)
    │   ├─ export.ts      # CSV / Excel / JSON / summary
    │   ├─ parser/csv.ts  # Papa Parse CSV pipeline
    │   ├─ pdf/parse-pdf.ts # PDF.js extraction
    │   ├─ pdf/log-history.ts # Admin log history table parser
    │   ├─ normalization/normalize.ts
    │   ├─ formatting/formatters.ts
    │   └─ insights/
    │       ├─ engine.ts   # analyzeRecords orchestrator
    │       ├─ lifecycle.ts # Per-listing journeys
    │       ├─ ordering.ts  # Chronological sort
    │       └─ status-tracker.ts
    └─ components/
        ├─ app-root.tsx   # View router
        ├─ shared/        # App shell + UI primitives
        ├─ upload/        # Upload & processing screens
        ├─ dashboard/
        ├─ records/       # Table, detail, compare, raw
        ├─ timeline/
        ├─ status/
        └─ insights/      # Insights + QA views
```

## 5. Technology Stack

| Layer      | Technology                     | Role                                               |
|------------|--------------------------------|----------------------------------------------------|
| Framework  | Next.js 15.4                   | App router, SSR shell, static assets               |
| UI         | React 19                       | Component tree and client interactivity            |
| Language   | TypeScript 5.9                 | Typed domain model end-to-end                      |
| Styling    | Tailwind CSS 4                 | Utility styling + OpenSooq CSS variables           |
| Fonts      | Cairo (display) + Inter (body) | Arabic/Latin brand typography                      |
| State      | Zustand 5                      | Global analysis state and view routing             |
| Tables     | TanStack Table 8               | Records grid, pagination, columns                  |
| Charts     | Recharts 3                     | Dashboard visualizations                           |
| CSV        | Papa Parse 5                   | Client-side CSV parsing                            |
| PDF        | pdfjs-dist 5                   | Text extraction from CDC PDFs                      |
| Excel      | SheetJS (xlsx)                 | Excel export                                       |
| Icons      | lucide-react                   | UI iconography                                     |
| Dates      | date-fns 4                     | Date helpers                                       |
| Validation | Zod 4                          | Available for schema work (custom parsers primary) |

### Runtime requirement

Node.js 20 or newer is required for local development and production builds.

## 6. Getting Started

---

### 6.1 Install & run (development)

```
npm install
npm run dev
```

Open `http://localhost:3000` . The dev server uses `WATCHPACK_POLLING=true` for reliable file watching in containerized environments.

### 6.2 Try the sample file

Use `sample-data/sample-cdc.csv` from the upload screen. It contains 16 CDC rows across 6 listings covering New → Pending → Active → Deactivated cycles, Premium status, Blocked violations, Hidden listings without price, and Expired listings.

### 6.3 Production build

```
npm run build
npm start
npm run lint
```



## 7. User Guide — Features & Screens

---

### 7.1 Upload

- Drag-and-drop or file picker for **PDF** and **CSV**
- Local-processing badge: files never leave the browser
- Errors include a local Error ID (e.g. `ERR-XXXXXX`) for support reference

### 7.2 Processing

Progress bar (0–100%) with a six-step checklist. Message confirms local browser processing.

### 7.3 App shell (post-analysis)

- **Header** — OpenSooq logo, global search (200ms debounce), file meta, sensitive-data toggle, export shortcut, “Upload new”
- **Sidebar** — Dashboard, Records, Timeline, Listing Status, Insights, QA Analysis, Raw Data (with counts / severity tones)
- **Coverage panel** — records, listings, fields, source type
- **Footer** — “OpenSooq CDC Analyzer · Made by Hashem Matouq”

### 7.4 Dashboard

- Six clickable stat cards (total / new / updated / deleted / violations / active)
- Status tracker summary with bucket chips
- Smart insights + top important events
- QA warning banner when issues exist
- Charts: operations pie, country bar, activity by hour, records over time
- Processing warnings list

### 7.5 Records

- TanStack Table with page sizes 25 / 50 / 100 / 250
- Human / Raw toggle
- Filters: operation, status, active, country, language, violation, featured, date range, full-text search
- Column visibility picker
- Export CSV, Excel, JSON, or Summary (filtered set)
- Select up to two rows to Compare; Open for Record Detail

### 7.6 Record Detail

- Badges for operation, status, active, featured, violation, first live, timestamp
- “Explain this record” plain-language bullets
- Listing journey mini-timeline from lifecycle data

- Human view: fields by category, Last Edit Action JSON, technical details
- Raw view: all original fields; field-help popovers with meaning

## 7.7 Timeline

- Modes: All activity | By listing
- Filters: Key events | Everything | Issues only
- Span cards (start, end, duration, event count)
- Day-grouped events with sticky headers; pagination (120/page)
- Per-listing lifecycle cards with expand/collapse and evidence

## 7.8 Listing Status

- Stats: listings tracked, changed, frequent changers (3+)
- Status bucket grid with filters and deep-link to records
- Search + filters (all / changed / 3+ changes); match any point vs current only
- Expandable status journey chips and log-history source badges

## 7.9 Insights & QA

- **Insights** — activity mix, violations, top country, peak hour; first-vs-last change groups
- **QA Analysis** — multi-status churn, update-after-delete, duplicate bursts, active without price, active+blocked/expired, featured+inactive

## 7.10 Compare & Raw Data

- **Compare** — side-by-side old/new; toggle “only changes”
- **Raw Data** — full extracted table, sticky header, incremental load (100 then +200)

## 8. Data Pipeline

### 8.1 CSV parser ( `lib/parser/csv.ts` )

- Library: Papa Parse
- Requires a header row; empty lines skipped
- APIs: `parseCsvText` , `parseCsvFile`
- Throws `NO_CDC_DATA` when no usable rows are found

### 8.2 PDF parser ( `lib/pdf/parse-pdf.ts` )

Uses `pdfjs-dist` with a local worker. Pages are extracted via `getTextContent` , then laid out by Y-baseline grouping.

#### Extraction strategies (in order):

1. Log history admin table ( `parseLogHistory` )
2. Delimited tables (headers with CDC hints such as `SF_OP` , `SF_TIMESTAMP` )
3. Key-value blocks ( `FIELD: value` / `FIELD = value` )
4. Inline JSON objects with  $\geq 3$  keys

#### Scanned PDFs

If extracted text is shorter than  $\sim 40$  characters  $\times$  page count, the file is flagged `scannedLikely` and processing throws `SCANNED_PDF` . Image-only scans are not supported.

### 8.3 Log history ( `lib/pdf/log-history.ts` )

Expected columns: Name | Is Moderator | Action | Action On | Diff | Date.

- Detects listing ID from `/post/{id}` URLs or labelled IDs
- Normalizes timestamps (ISO, DMY, YMD)
- Maps action text to status definitions; unresolved rows produce warnings
- Emits synthetic CDC rows with `SOURCE: LOG_HISTORY` , merged with table records

### 8.4 Normalization ( `lib/normalization/normalize.ts` )

`recordsFromRows`  $\rightarrow$  `normalizeRawRow` for each non-empty row:

1. Uppercase keys via `normalizeKey()`
2. Resolve aliases (e.g. `ID` / `LISTING_ID` / `POST_ID` ; `SF_OP` / `OPERATION` )
3. Parse `LAST_EDIT_ACTION` JSON when present
4. Build nested `CDCRecord` (entity / status / user / cdc / raw)
5. Assign `uid` : `{id}-row-{index}-{requestId|index}`

**Operation mapping:** insert ( `i` , create), update ( `u` , updated), delete ( `d` , deleted), else unknown.

**Timestamp priority:** SF\_TIMESTAMP → TIMESTAMP → RECORD\_UPDATE\_DATE → DATE → RECORD\_INSERT\_DATE .

## 8.5 Chronological ordering ( `ordering.ts` )

CDC dumps are often newest-first and timestamps may share second precision. Sort priority:

1. SF\_OFFSET\_ID / SF\_OFFSET / OFFSET\_ID / LAST\_EVENT\_ID (ascending)
2. SF\_TIMESTAMP (ascending)
3. Original file index as tie-break

## 9. Field Dictionary

Defined in `src/config/field-definitions.ts`. Unknown fields auto-generate a “Technical Information” fallback via `getFieldDefinition`.

| Category              | Example fields                                                                                                                                                 | Notes                         |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Listing Information   | ID, TITLE, DESCRIPTION, PRICE, BASE_PRICE, PRICE_CURRENCY, HAS_PRICE, CATEGORIES_ID, NEW_CAT_ID, CITIES_ID, COUNTRIES_ID, LANG, GEO_LAT, GEO_LNG, DISPLAY_NAME | Core listing attributes       |
| Listing Status        | ACTIVE, STATUS, IS_FEATURED, IS_HIGHLIGHTED, VIOLATION, IS_FIRST_LIVE, RECORD_EXPIRATION_DATE                                                                  | Visibility & moderation state |
| CDC                   | SF_OP, SF_TIMESTAMP, SF_OFFSET_ID, REQUEST_ID, LAST_EDIT_ACTION, RECORD_INSERT_DATE, RECORD_UPDATE_DATE                                                        | Change-stream metadata        |
| User Information      | MEMBERS_ID, EMAIL, PHONE                                                                                                                                       | Marked <b>sensitive</b>       |
| Activity              | NUMBER_OF_TIMES_REPOSTED, POST_SOURCE, POST_TYPE                                                                                                               | Repost & channel signals      |
| Technical Information | POST_INSERT_IP_COUNTRY, POST_UPDATE_IP_COUNTRY                                                                                                                 | Geo of client actions         |

Attached formatters include: `price`, `yesNo`, `status`, `violation`, `operation`, `datetime`, `country`, `language`, `email`, `phone`, `jsonAction`.

Default visible table columns are listed in `DEFAULT_TABLE_COLUMNS`.

## 10. Listing Status Codes

Official OpenSooq STATUS codes live in `src/config/listing-statuses.ts`. Text aliases resolve log-history labels and informal exports. Normalization lowercases, strips punctuation, and supports Arabic Unicode.

| Code | Label       | Tone    | Flags     | Description                      |
|------|-------------|---------|-----------|----------------------------------|
| 100  | New         | neutral | —         | Just created, not yet reviewed   |
| 101  | Pending     | warning | —         | Awaiting approval / moderation   |
| 102  | Deactivated | neutral | —         | Manually turned off (reversible) |
| 103  | Blocked     | danger  | isProblem | Suspended for policy / violation |
| 104  | Hidden      | neutral | —         | Not publicly visible             |
| 200  | Expired     | warning | isProblem | Past valid lifetime              |
| 300  | Active      | success | isLive    | Live and fully visible           |
| 301  | Premium     | brand   | isLive    | Active with paid features        |

### Helper sets

```
LIVE_STATUS_CODES = {300, 301} · PROBLEM_STATUS_CODES = {103, 200} · Helpers: isLiveStatus ,  
isProblemStatus , listingStatusLabel , listingStatusTone
```

# 11. Insights, Lifecycle & QA

---

## 11.1 Analysis orchestrator

`analyzeRecords` in `lib/insights/engine.ts` runs:

1. `computeStats` — operation / violation / active counts
2. `buildLifecycles` — per-listing journeys
3. `buildGlobalTimeline` — flat sorted steps
4. `buildStatusTracker` — status history
5. `buildSmartInsights` — activity mix, violations, top country, peak hour
6. `buildQaInsights` — anomaly rules
7. `detectImportantEvents` — rule actions, violations, deletes (cap 100)
8. `buildTimeline` — flat CDC events
9. `detectChanges` — first-vs-last field diffs (cap 200 groups)
10. `buildCharts` — operations, status, country, language, overTime, violationsOverTime, activityByHour
11. Warnings for unrecognized status codes

## 11.2 Lifecycle

Listings are grouped by ID, sorted chronologically, and consecutive pairs are diffed across 18 tracked fields (title, description, price, status, active, featured, violation, category, city, country, language, hasPrice, repost count, expiration, etc.).

Each step has a `stepKind`: created, activated, deactivated, updated, price, status, violation, rule, featured, category, reposted, expired, deleted, or event.

## 11.3 Status tracker

- Groups by listing ID, else title+member, else row index
- Builds `StatusPoint[]` and `StatusTransition[]`
- Aggregates buckets with current / ever / record counts
- `MULTI_CHANGE_THRESHOLD = 3` drives warning severity
- Potential issues: active + problem status; featured + inactive

## 11.4 Evidence categories

| Category        | Meaning                                      |
|-----------------|----------------------------------------------|
| FACT            | Directly observed from field values          |
| DERIVED         | Computed from aggregates or diffs            |
| INTERPRETATION  | Inferred from rule-engine / action JSON      |
| POTENTIAL_ISSUE | QA anomaly — not labelled as a confirmed bug |



## 12. State Management

Global store: `useAppStore` in `src/lib/store.ts` (Zustand).

### 12.1 State shape

| Field                            | Type                               | Default / notes                    |
|----------------------------------|------------------------------------|------------------------------------|
| <code>view</code>                | <code>AppView</code>               | <code>"upload"</code>              |
| <code>processingSteps</code>     | <code>ProcessingStep[]</code>      | Six pending steps                  |
| <code>progress</code>            | <code>number</code>                | 0–100                              |
| <code>error / errorId</code>     | <code>string   null</code>         | User-facing error + local ID       |
| <code>analysis</code>            | <code>AnalysisResult   null</code> | Full analysis payload              |
| <code>filters</code>             | <code>FilterState</code>           | Search + facet filters             |
| <code>viewMode</code>            | <code>human   raw</code>           | Default human                      |
| <code>showSensitive</code>       | <code>boolean</code>               | Default false (masked)             |
| <code>selectedRecordUid</code>   | <code>string   null</code>         | Opens record-detail                |
| <code>compareA / compareB</code> | <code>string   null</code>         | Compare selection                  |
| <code>visibleColumns</code>      | <code>string[]</code>              | <code>DEFAULT_TABLE_COLUMNS</code> |

### 12.2 Key actions

- `processFile(file)` — full parse + analyze pipeline
- `setView`, `setFilters`, `resetFilters`
- `setViewMode`, `setShowSensitive`
- `setSelectedRecord`, `setCompare`, `setVisibleColumns`
- `clearData` — return to upload state

Client filtering is performed by `filterRecords(records, filters)` across ID, title, member, phone, requestId, and raw fields.

## 13. Export & Formatting

### 13.1 Exports ( `lib/export.ts` )

| Function                      | Format | Content                                  |
|-------------------------------|--------|------------------------------------------|
| <code>exportCsv</code>        | .csv   | Raw field values, properly escaped       |
| <code>exportExcel</code>      | .xlsx  | Sheet "CDC" via SheetJS                  |
| <code>exportJson</code>       | .json  | Array of <code>record.raw</code> objects |
| <code>exportPdfSummary</code> | .txt   | Plain-text summary of filtered counts    |

Downloads use `URL.createObjectURL`. Exports apply to the currently filtered Records set.

### 13.2 Masking & display formatters

| Function                  | Behavior when sensitive hidden            |
|---------------------------|-------------------------------------------|
| <code>maskPhone</code>    | Only last 4 digits visible                |
| <code>maskEmail</code>    | First character + <code>***@domain</code> |
| <code>maskIp</code>       | Partial octets hidden                     |
| <code>maskMemberId</code> | First 2 + <code>***</code> + last 2       |

Country codes expand to names (JO → Jordan, AE → UAE, SA → Saudi Arabia, ...). Languages map to Arabic, English, French, Kurdish. Operations become Created / Updated / Deleted. Datetimes use en-GB locale with 12-hour time. Rule actions are humanized (e.g. activate → "Listing activated"; actors: owner, admin, system, rule\_engine, moderator).

## 14. Privacy & Security

---

### Default posture

Uploaded files are read with browser File APIs and analyzed in memory. Nothing is uploaded to OpenSooq servers or third-party AI services as part of the analysis flow.

- Sensitive fields (member ID, email, phone) masked unless the user toggles reveal
- Error IDs are generated locally for support correlation only
- No environment secrets are required for core functionality
- Suitable for handling internal CDC exports on a locked-down workstation

### Operational note

Revealing sensitive data on a shared screen still exposes PII. Prefer keeping masking on during demos and screen shares.

## 15. Sample Data

File: `sample-data/sample-cdc.csv` — 16 data rows, 29 columns, 6 unique listing IDs.

| Listing ID | Title                | Journey highlights                                                         |
|------------|----------------------|----------------------------------------------------------------------------|
| 284387206  | أجمل الأرقام رباعي   | New → Pending → Active → Deactivated → Active with over-limit rule actions |
| 284387300  | iPhone 14 Pro Max    | Insert → price increase → Premium (301)                                    |
| 284387401  | Villa for rent Amman | Blocked (103) with violation + moderator action                            |
| 284387500  | Laptop Dell XPS      | Hidden (104), no price ( HAS_PRICE=0 )                                     |
| 284387600  | Old listing          | Expired (200)                                                              |
| 284387900  | Moderation journey   | Five status changes: New → Pending → Active → Deactivated → Active         |

Notable columns: `SF_OFFSET_ID` for ordering, `LAST_EDIT_ACTION` JSON, sensitive phone/email, `POST_INSERT_IP_COUNTRY` .

## 16. Type System Reference

Primary contracts live in `src/types/cdc.ts`.

### 16.1 Key unions

- `CDCOperation`: insert | update | delete | unknown
- `InsightSeverity`: information | warning | potential\_issue
- `EvidenceCategory`: FACT | DERIVED | INTERPRETATION | POTENTIAL\_ISSUE
- `ViewMode`: human | raw
- `AppView`: upload, processing, dashboard, records, timeline, status, insights, qa, raw, compare, record-detail

### 16.2 Core interfaces (summary)

| Interface                                                  | Purpose                                                  |
|------------------------------------------------------------|----------------------------------------------------------|
| <code>CDCRecord</code>                                     | Normalized listing event with entity/status/user/cdc/raw |
| <code>Lifecycle</code> / <code>LifecycleStep</code>        | Per-listing journey and individual steps                 |
| <code>ListingStatusTrack</code>                            | Full status history for one listing                      |
| <code>StatusTrackerResult</code>                           | Buckets + tracks + totals                                |
| <code>InsightItem</code> / <code>ImportantEvent</code>     | Smart insights and highlighted events                    |
| <code>FieldChange</code> / <code>ListingChangeGroup</code> | First-vs-last diffs                                      |
| <code>DashboardStats</code>                                | Aggregate operation/violation counts                     |
| <code>AnalysisResult</code>                                | Complete analysis payload held in the store              |
| <code>FilterState</code>                                   | UI filter facets                                         |
| <code>SourceFileMeta</code>                                | Uploaded file metadata                                   |

## 17. Build, Deploy & Operations

---

### 17.1 Scripts

| Script | Command                    | Purpose                                    |
|--------|----------------------------|--------------------------------------------|
| dev    | <code>npm run dev</code>   | Local development server                   |
| build  | <code>npm run build</code> | Production Next.js build                   |
| start  | <code>npm start</code>     | Serve production build                     |
| lint   | <code>npm run lint</code>  | ESLint (Next core-web-vitals + TypeScript) |

### 17.2 Deployment model

- Standard Next.js hosting (Vercel, Node `next start`, or container)
- No backend API required for CDC analysis
- No environment variables required for core features
- PDF.js worker is bundled from the package via `import.meta.url`
- `next.config.ts` disables the canvas alias used by some PDF.js builds

### 17.3 Branding & metadata

- Title: OpenSooq CDC Data Analyzer
- Author metadata: Hashem Matouq
- Favicon / logo: `/opensooq-logo.png`
- CSS tokens: `--os-blue`, `--os-green`, `--os-navy`, and supporting soft/border/muted scales

## 18. Troubleshooting

| Symptom                        | Likely cause                                             | What to try                                                        |
|--------------------------------|----------------------------------------------------------|--------------------------------------------------------------------|
| Error <code>SCANNED_PDF</code> | Image-only / OCR-less PDF                                | Export a text-based PDF or use CSV instead                         |
| Error <code>NO_CDC_DATA</code> | Empty file or no header/rows                             | Confirm delimiter and header row; try sample CSV                   |
| Events appear out of order     | Missing <code>SF_OFFSET_ID</code> ; colliding timestamps | Include offset IDs in the export; verify sort priority             |
| Unknown status labels          | Nonstandard status text                                  | Add alias in <code>listing-statuses.ts</code> or use numeric codes |
| Sensitive values visible       | Reveal toggle enabled                                    | Turn off “Show sensitive” in the header                            |
| Large file feels slow          | Browser memory / CPU bound                               | Filter the export upstream; process fewer pages/rows               |
| PDF tables incomplete          | Complex layout / multi-column gaps                       | Prefer CSV; or ensure tab/pipe-delimited text layout               |

# 19. Appendix — Glossary & Credits

## 19.1 Glossary

| Term             | Definition                                                                    |
|------------------|-------------------------------------------------------------------------------|
| CDC              | Change Data Capture — stream of insert/update/delete events from a data store |
| Listing          | An OpenSooq classified advertisement identified primarily by ID               |
| Lifecycle        | Ordered reconstruction of a listing's events from first to last               |
| SF_OP            | CDC operation field (insert / update / delete)                                |
| SF_OFFSET_ID     | Monotonic stream offset used for authoritative event ordering                 |
| LAST_EDIT_ACTION | JSON payload describing who/what changed the listing and why                  |
| Log history      | Admin moderation table sometimes embedded in PDF exports                      |
| Evidence         | Supporting values or rules shown with each insight or timeline step           |

## 19.2 Document control

| Item            | Value                                              |
|-----------------|----------------------------------------------------|
| Product         | OpenSooq CDC Data Analyzer (opensooq-cdc-analyzer) |
| Version covered | 0.1.0                                              |
| Author          | Hashem Matouq                                      |
| Organization    | OpenSooq                                           |
| Document type   | Full technical & user documentation (PDF)          |
| Generated       | August 2026                                        |

OpenSooq CDC Data Analyzer — Convert complicated CDC / raw technical data into clear, readable business information. Made by Hashem Matouq. Brand colors: OpenSooq blue #33529D, OpenSooq green #87BE57. This document describes the application as implemented in the project repository and is intended for internal engineering, QA, and operations use.